



HASHDIT

HashDit

Smart Contract Security

ListaDao AtlasMultiFeedAdaptor **Security Audit Report**

ListaDao

14 Sep 2026 – 16 Sep 2026

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Period	14 Sep 2026 - 16 Sep 2026
Overall Risk	Informational
Project Name	Listado AtlasMultiFeedAdaptor Audit
Github	https://github.com/lista-dao/lista-new-contracts/pull/66
Commit	c03aba37b7a6ca95405b3cc32c50ba239712ca0c

No.	Contract Name	Verdict	Details
1	IAtlasMultiFeed.sol	Informational	[I04]
2	AtlasMultiFeedAdaptor. sol	Informational	[I01]
			[I02]
			[I03]
			[I04]
			[I05]

Findings

[I01] Intended single-oracle config performs no price sanity check

Contract AtlasMultiFeedAdaptor.sol

Severity Informational

Description

The ResilientOracle._getMainOraclePrice() function returns the main oracle price with validated = true and skips bound validation whenever no pivot oracle is enabled. Both the fork test and the bStock design rely on exactly this configuration, using enableFlagsForOracles = [true,false,false].

JavaScript

```
function _getMainOraclePrice(
    address asset,
    uint256 pivotPrice,
    bool pivotEnabled
) internal view returns (uint256, bool) {
    (address mainOracle, bool mainOracleEnabled) = getOracle(asset,
OracleRole.MAIN);
    if (mainOracleEnabled && mainOracle != address(0)) {
        try this.getPriceFromOracle(
            mainOracle,
            tokenConfigs[asset].timeDeltaTolerance
        ) returns (uint256 mainOraclePrice) {
            if (!pivotEnabled) {
                return (mainOraclePrice, true);
            }
            if (pivotPrice == INVALID_PRICE) {
                return (mainOraclePrice, false);
            }
            return (
                mainOraclePrice,
                boundValidator.validatePriceWithAnchorPrice(asset,
mainOraclePrice, pivotPrice)
            );
        } catch {
            return (INVALID_PRICE, false);
        }
    }

    return (INVALID_PRICE, false);
}
```

Description

Impact

	In this setup the market fully trusts the Atlas price without any cross-source anchor to compare it against. There is therefore no second line of defense against a wrong feedId or a manipulated or erroneous source value, because BoundValidator never runs and no other oracle is consulted to catch a gross deviation.
Recommendation	In the future, where feasible, configure a pivot or fallback oracle, such as a Chainlink feed or a DEX anchor, so that BoundValidator can compare the main price against an independent source and reject values that deviate grossly.
Status	[Open / Fixed / Acknowledged]

[I02] Stale pre-halt price served as live at reopen when the halt is shorter than the freshness tolerance

Contract	AtlasMultiFeedAdaptor.sol
-----------------	---------------------------

Severity	Informational
-----------------	----------------------

Description	Description In ResilientOracle, updatedAt is set to aggregatedTs, and the freshness gate rejects a price when $\text{block.timestamp} - \text{aggregatedTs} > \text{tolerance}$ <pre> JavaScript function getPriceFromOracle(address oracle, uint256 tolerance) external view returns (uint256) { try AggregatorV3Interface(oracle).latestRoundData() returns (uint80, int256 answer, uint256, uint256 updatedAt, uint80) { if (tolerance != 0 && block.timestamp - updatedAt > tolerance) { return INVALID_PRICE; } return uint256(answer); } catch { return INVALID_PRICE; } } </pre> Impact Because of this, the behavior at the instant a market switch reopens falls into one of two regimes depending purely on the age of aggregatedTs relative to the tolerance. 1. In the first regime, the market close could have lasted longer than the tolerance and no fresh push has landed yet. Here aggregatedTs is older than the tolerance, so getPriceFromOracle returns 0 and peek reverts with "invalid resilient oracle price". The market is open according to the switch but is
--------------------	---

unusable until the first fresh Atlas aggregation arrives. This fails closed, but borrow and liquidation both revert during this window, and the revert reason is the ResilientOracle one rather than StockMarketClosed. Monitoring that only watches the switch will therefore misread the state.

2. In the second and more dangerous regime, the market close could have been shorter than the tolerance, for example a brief intraday or volatility halt combined with a tolerance of 300s+. The pre-close aggregatedTs is still within the tolerance, so the stale pre-close price is served as "live" from the moment of reopen until a new push replaces it. If the underlying asset gapped during the halt, which is usually the reason it was halted, positions can be borrowed against or liquidated at the pre-halt price throughout that window. Which regime occurs is determined entirely by the age of aggregatedTs versus the tolerance.

Note that this was also present in the previous AtlasOracleAdaptor.sol, Consider the following:

1. Reopen only after a fresh post-halt push (operational, most reliable): flip setGlobal(true)/open only once a new aggregation with aggregatedTs after the halt has landed. This forces the oracle to wait for a genuine post-reopen push in both regimes and closes the window where positions could be priced on a stale pre-halt value.
2. Set timeDeltaTolerance below the shortest halt you care about (ResilientOracle owner): forces meaningful closes to fail closed. Trade-off: too tight causes spurious reverts during normal push gaps, and it still misses a halt shorter than the tolerance where the underlying gapped.
3. Enable a pivot plus BoundValidator as a cross-source anchor that rejects a stale price that gapped during the halt, when such an anchor exists.
4. Monitor for the invalid resilient oracle price revert, not just StockMarketClosed, since the fail-closed regime surfaces the ResilientOracle reason.

Recommendation

Status [Open / Fixed / Acknowledged]

[I03] Considerations for migration to AtlasMultiFeedAdaptor from AtlasOracleAdaptor

Contract AtlasMultiFeedAdaptor.sol

Severity **Informational**

Description
Existing bStock markets are priced through the ResilientOracle, which holds one oracle address per asset. Migrating an asset from the legacy AtlasOracleAdaptor to the AtlasMultiFeedAdaptor is performed by repointing that asset's oracle slot with a

privileged governance call to setOracle. Two properties of the price source change during this swap, and neither is validated on chain.

1. The binding between the asset and its price becomes unverifiable. The legacy adaptor wraps a dedicated Atlas feed contract for a single asset, whereas the new adaptor identifies its price by an immutable feedId inside a shared registry. Nothing on chain proves that the chosen feedId corresponds to the same asset the old adaptor priced. A wrong or mistyped feedId still returns a valid and fresh price, but for a different instrument.
2. The price freshness basis changes. The legacy adaptor passes the feed's updatedAt through unchanged, which reflects the on chain push time. The new adaptor reports aggregatedTs, the off chain aggregation time, which is always equal to or older than the push time. Under the same timeDeltaTolerance, the new adaptor is therefore equal to or stricter about staleness, so a snapshot that was considered fresh before the swap can be treated as stale afterward.

Impact

Because the setOracle swap performs no equivalence check between the outgoing and incoming price sources, the migration can move the reported price, or its freshness, for the asset at the instant it lands.

- If the new price is lower than the old one, open positions can be forced into mass liquidation.
- If it is higher, positions can borrow against an inflated valuation. If the aggregatedTs age already exceeds the configured tolerance, peek reverts and the market freezes.

Recommendation

Consider ensuring that the multi feed provides accurate prices within a certain threshold of the previous single feed contract before performing the required migration.

Status

[Open / Fixed / Acknowledged]

[I04] isOpenRead() flag Atlas integration risks

Contract

AtlasMultiFeedAdaptor.sol & IAdapterMultiFeed.sol

Severity

Informational

Description

Description

The adaptor includes isOpenRead() in its subset of the IAtlasMultiFeed interface, yet the internal _read() path never calls it.

JavaScript

```
function isOpenRead() external view returns (bool);
```

Per [Atlas documentation](#), `isOpenRead()` reports whether open read mode is currently active. When it is true, any caller can read prices. When it is false, only whitelisted callers and regular EOAs that have not been delegated via ERC-7702 are permitted. The flag therefore controls who may read, not whether the stored price is valid or whether the underlying market is open.

Impact

Given that meaning, omitting the check inside `_read()` does not cause the adaptor to serve invalid or not ready data. Read access is enforced by `fetch()` itself, which reverts when the caller is not authorized.

Adding a pre check on `isOpenRead()` would in fact be incorrect, because the adaptor can be whitelisted while the flag is false, and a naive revert on a false value would wrongly block a legitimately authorized read.

The one real consequence is liveness. If the registry switches to whitelist mode while the adaptor address is not authorized, `fetch()` reverts, the ResilientOracle treats the result as an invalid price, and the dependent markets fail closed until access is restored.

No fix required, given `fetch()` already enforces access and the deployment script correctly uses the flag as a deployment precondition, so keep it in the interface for that purpose

Recommendation

Consider documenting this risk for bStock market integration and ensure the adaptor address is added to the source whitelist before any transition to whitelist mode to prevent blockage of market actions on the Moolah side.

Status

[Open / Fixed / Acknowledged]

[I05] Truncation floor and uint80 price ceiling are undocumented in AtlasMultiFeedAdaptor

Contract AtlasMultiFeedAdaptor.sol

Severity Informational

Description
AtlasMultiFeedAdaptor converts an 18 decimal snapshot price to 8 decimals by dividing by $1e10$, truncating toward zero, and rejects the result as invalid when it rounds to 0. This is the same conversion behavior as AtlasOracleAdaptor, so any underlying price below $1e10$ (approximately $\$1e-8$ USD at 18 decimals) truncates to 0. In AtlasMultiFeedAdaptor this then reverts as an invalid price, meaning a legitimate but very small non zero price would be silently treated as unusable.

JavaScript

```
function _read() internal view returns (int256 answer, uint256
updatedAt) {
    // Atlas may upgrade its source proxy; never silently apply the
    wrong scale.
    if (multiFeed.decimals() != 18) revert InvalidDecimals();
    IAtlasMultiFeed.PriceSnapshot memory snapshot =
multiFeed.fetch(feedId);
    uint256 scaledPrice = uint256(snapshot.price) / SCALE_DIVISOR;
    if (scaledPrice == 0) revert InvalidPrice();
    if (
        snapshot.aggregatedTs == 0 ||
        snapshot.onchainTs == 0 ||
        snapshot.aggregatedTs > snapshot.onchainTs ||
        snapshot.onchainTs > block.timestamp
    ) revert InvalidTimestamp();

    // uint80 / 1e10 is always safely representable as int256.
    return (int256(scaledPrice), uint256(snapshot.aggregatedTs));
}
```

At the high end, the snapshot price field is a uint80, so the source can carry a raw 18 decimal price no larger than $2^{80} - 1$, which is roughly $1.2e24$ at 18 decimals, or about \$1.2 million per unit. Prices above that ceiling cannot be represented by the snapshot at all. This is a limitation unique to the multi feed design, since AtlasOracleAdaptor reads an int256 answer that has no comparable ceiling, and it is not documented anywhere in AtlasMultiFeedAdaptor.

Impact

AtlasOracleAdaptor carries an explicit note describing this exact risk, that the conversion truncates toward zero, that answers below $1e10$ round down to 0 and are treated as INVALID_PRICE, that this is safe for the current asset list, and that operators must revisit the math before listing ultra low priced feeds where truncation could mask a legitimate non zero price.

AtlasMultiFeedAdaptor only mentions that prices are rounded down and omits this warning entirely. Because the multi feed adaptor is the one being deployed per asset, the missing note means an operator onboarding a new low priced feed through it gets no indication of the truncation floor or the precondition for safely listing such an asset, even though the underlying risk is identical.

Additionally, because the multi feed adaptor is the one deployed per asset, an operator onboarding a new feed through it receives no indication of either the truncation floor or the uint80 ceiling, even though listing an asset whose real price falls below $1e10$ or above the uint80 maximum would cause the adaptor to reject or fail to represent a legitimate price.

Recommendation	Consider Document both ends of the representable price range in AtlasMultiFeedAdaptor by including similar documentation related to the truncation-to-zero note from AtlasOracleAdaptor into AtlasMultiFeedAdaptor so both adaptors document the same constraint as well as the uint80 price limit for the new multifeed adaptor.
Status	[Open / Fixed / Acknowledged]